

Grundläggande syntax

Man behöver kunna en del grundläggande syntax innan man kan skriva C++-program. Nedan finns de grundläggande

Separering av satser

En *sats* i C++ kan ses som en instruktion, funktionsanrop, tilldelning eller annat som utför något i C++. Normalt skriver man en sats per rad för att hålla program läsbara, men man kan även kombinera flera satser på samma rad. Satser separeras alltid med ett semikolon (;), även om man har en sats per rad. I vissa fall kan man kombinera ihop satser med hjälp av ett kommatecken (,), men dylika fall behandlas inte i detta kompendium. Efter tecken som { och } använder man inte semikolon, inte heller efter rader som börjar med #. Dylika rader hör egentligen inte till C++, och kommer aldrig att ses av kompilatorn. De är s.k. *preprocessorkommandon* som behandlas före filen egentligen kompileras (se [Kapitel 10](#)). Kommentarer behöver inte heller avslutas med semikolon (se [avsnittet Kommentarer](#)). Efter vissa nyckelord behöver skall man inte heller placera ett semikolon, eftersom den efterföljande raden hör ihop med raden med nyckelordet. Hur satser separeras är enkelt, och de exempel som finns i detta kompendium belyser alla olika möjligheter för konstruktion av satser.

Kodblock

I C++ använder man sig av { och } för att begränsa ett kodblock. Ett kodblock är en mängd av noll eller flera satser som hör ihop på något sätt. Det kan t.ex. vara kod som utförs i en slinga eller hör till en funktion. I andra språk används t.ex. BEGIN och END för samma sak. Vi kommer att stöta på kodblock i varje program vi skriver.

Formatering av kod

Kod kan i C++ formateras nästan hur som helst. Man kan använda tomma rader och indentering för att göra koden lättare läsbar och mera "luftig", men det är inget tvång. De flesta editorer som används för redigering av C++ stöder automatisk indentering av koden så att man enkelt kan visualisera olika kodblock, men det är ingenting som tvingar en att skriva på det sättet. Vi kunde lika väl ha skrivit det på följande sätt:

```
#include <iostream>
int main(){cout<<"Hello world!"<<endl;}
```

Vilken man föredrar hoppas jag är självklart. Personligen föredrar jag luftig och indenterad kod som är försedd med rikligt med kommentarer (se [avsnittet Kommentarer](#)), speciellt i inlämningsuppgifter (hint, hint). För kompilatorn är det ingen skillnad hur koden är formaterad, så länge som syntaxen är korrekt. I vissa fall måste man lämna tomrum mellan tecken för att undvika att kompilatorn gör feltolkningar.

Undvik alltså att skriva obskyr kod. Du har lättare att själv läsa din egen kod om några veckor, månader eller år om du behödat dig om att göra den läslig. Du kanske tänker att "inte läser jag min kod någonsin mera" eller "jag förstår den nog". Dylika tankar ledde till att Cobol-programmerare för

20 år sedan tänkte "äh, detta program används inte mera år 2000, jag kan skriva årtalen med två siffror". Vi ser hur det gick. Program har en tendens att överleva sin tilltänkta livslängd. Ofta vill man senare se i sina gamla program hur man löst ett visst problem, hur en algoritm såg ut eller hur syntaxen för någonting fungerar.

Kommentarer

Man kan *kommentera* kod i C++. Med kommentering menas en fri text som skrivs in i programmet och som på något sätt berättar vad programmet gör just på det stället. Kompilatorn ignorerar alla kommentarer i programmet, så ett körbart program blir inte större av att man kommenterar ordentligt. Kommentarer är en form av livförsäkring då man senare skall försöka förstå vad ett program gör. Normalt tänker man "klart jag vet vad mitt program gör, jag har ju skrivit det". Det kanske stämmer så länge man skriver programmet och allt är i färskt minne, men gör man det efter två månader eller två år?

En kommentar i C++ kan se ut på två olika sätt, beroende på dess längd. Det normala sättet är att använda C++-kommentarer som börjar med två /. T.ex. följande innehåller några kommentarer:

```
// ett enkelt exempel-program för att visa hur man kan
// utföra den utskriften av texten "Hello world!"
// till skärmen
int main () {
    // skriv ut vårt meddelande
    cout << "Hello world!" << endl; // vår text
}
```

Kommentarer kan börja var som helst på en rad, men all text *efter* // till radens slut tolkas av kompilatorn som en kommentar. Man kan på så vis enkelt kommentera ut kod som man inte vill använda. Om kommentaren fortsätter på flera rader måste ett nytt // placeras på varje rad. Om man har långa kommentarer som fortsätter över flera rader kan man använda *block-kommentarer*, som C++ lånat från C. Vi kan skriva om vårt program men C-kommentarer:

```
/* ett enkelt exempel-program för att visa hur man kan
   utföra den utskriften av texten "Hello world!"
   till skärmen */
int main () {
    /* skriv ut vårt meddelande */
    cout << "Hello world!" << endl; /* vår text */
}
```

C-kommentarer har en explicita start- och slutmarkeringar (/*) respektive */. Båda måste finnas för att det skall vara en korrekt kommentar. C-kommentarer kan fortsätta över många rader, och används normalt för längre kommentarer, medan C++-kommentarer vanligen används för kortare kommentarer. Vilket sätt man använder spelar ingen roll, båda ignoreras ändå av kompilatorn, huvudsaken är att man kommenterar överhuvudtaget.

[Förutgående](#)

Ett första program

[Hem](#)

[Upp](#)

[Nästa](#)

Variabler och datatyper